Project 3 on Machine Learning

Three paths: differential equations with neural networks, a classification study with several methods,
or the statistics and mathematics behind the methods

Deadline December 11 (midnight), 2026

Data Analysis and Machine Learning FYS-STK3155/FYS4155
Department of Physics, University of Oslo, Norway

November 10, 2026

Deliverables

First, join a group in Canvas with your group partners. Pick an available group for Project 3 in the **People** page.

In Canvas, deliver as a group and include:

- A PDF of your report which follows the guidelines covered below. Additional requirements include:
  - It should be around 5000 words; use the word counter in Overleaf for this. This often corresponds to 10–12 pages. References and appendices are excluded from the word count. Appendices should read like regular text and can include additional calculations you have made.
  - It should include around 10–15 figures. You can include more figures in appendices and/or as supplemental material in your repository. Including additional figures in appendices requires also that you add text which explains the additional material.
- A comment linking to your GitHub repository (or a folder in one of your GitHub repositories) for this project. The repository must include
  - a PDF file of the report;
  - a folder named `Code`, where you put Python files for your functions and notebooks for reproducing your results. Remember to use a seed for generating random data and for train-test splits when generating final results;
  - a `README` file with the names of the group members, a short description of the project, a description of how to install the required packages to run your code (from a `requirements.txt` file or similar, such as a plain text description), and the names and descriptions of the various notebooks in the `Code` folder and the results they produce.

Preamble: note on writing reports, using reference material, AI and other tools

We want you to answer the three different projects by handing in reports written like a standard scientific/technical report. The links at https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/Projects contain more information. There you can find examples of previous reports, the projects themselves, how we grade reports etc. How to write

reports will also be discussed during the various lab sessions. Please do ask us if you are in doubt. In that folder you will find how we evaluate the projects. You will receive a feedback with a final score for each project.

When using codes and material from other sources, you should refer to these in the bibliography of your report, indicating wherefrom you for example got the code, whether this is from the lecture notes, software like **Scikit-Learn**, **JAX**, **TensorFlow**, **PyTorch** or other sources. These sources should always be cited correctly. How to cite some of the libraries is often indicated from their corresponding GitHub sites or websites; see for example how to cite Scikit-Learn at `https://scikit-learn.org/dev/about.html`.

Use of LLMs – note the updated rules. You may use Large Language models (LLMs) like ChatGPT, Claude or similar as a *supportive measure* when developing your code and when working through the material, that is, as a tool that helps you understand, check and improve work which is your own, and not as a substitute for doing the analysis, writing the code or drawing the conclusions yourself. The guidelines at `https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md` have been **updated since Project 2**, and the update concerns the *writing of the report*:

- For the **text of the report**, generative AI may only be used at *contribution level 1* of the guidelines: spell-checking, grammar and minor phrasing. Having an LLM draft, rewrite or restructure a section of the report, in whole or in part, is **not acceptable use** (Section 3.1 of the guidelines). The purpose of the projects is that *you* learn to analyze data and to write a scientific report about it; offloading the writing to a language model defeats that purpose, and undeclared or disallowed use will be treated as a breach of the University of Oslo's academic integrity rules.
- For the **code**, the contribution levels 0–4 of the guidelines still apply. Every LLM-assisted function or cell must carry the inline `LLM-assisted` tag described there, and must have been tested and understood by you.

In the report, you should after the conclusions write a declaration on how you have used LLMs, following the instructions in the guidelines: the tools used, the contribution level (0 or 1) for every section of the text, and the level for every code file or notebook. Failing to include the declaration ***leads to an automatic deduction of five (5) points out of the final score of 100 for the project***. Remember that *you* are responsible for every line of code and every claim in the report: be prepared to explain and defend all of it. If you use LLMs for your code, please take a look at the guidelines at `https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/UsingLLMs` on how to use these tools efficiently.

If you would like to study other data sets, feel free to propose other sets. What we have proposed here are mere suggestions from our side. If you opt for another data set, consider using a set which has been studied in the scientific literature. This makes it easier for you to compare and analyze your results. Comparing with existing results from the scientific literature is also an essential element of the scientific discussion. The University of California at Irvine with its Machine Learning repository at `https://archive.ics.uci.edu/ml/index.php` is an excellent site to look up for examples and inspiration. Kaggle.com is an equally interesting site. Feel free to explore these sites and/or ask us for inputs relevant to your discipline.

Project 3: choose one of three paths

Projects 1 and 2 prescribed the data and the methods. Project 3 is where you choose. We propose three paths, and you should pick **one** of them:

- **Path A: Solving differential equations with neural networks.** The one-dimensional

diffusion equation solved with an explicit finite-difference scheme and with neural networks, both with hard-coded boundary and initial conditions and as a physics-informed neural network (PINN), followed by a study of how the network complexity affects the result. Other ordinary or partial differential equations may replace the diffusion equation after discussing it with us. This is the path for those of you who are interested in the numerical solution of differential equations.

- **Path B: A classification study with several methods.** A multiclass classification problem on a data set which is *not* MNIST, analyzed with at least three of the methods of the course – for example gradient boosting, a feed-forward neural network and a convolutional neural network – compared on equal terms with the tools of Chapter 5 (confusion matrix, precision, recall, ROC curves and more) and with the published literature on the data set. Our default data set is Fashion-MNIST; you may replace it with a data set of your own choice.
- **Path C: The statistics and mathematics behind the methods.** For those of you who want to understand *why* the methods of the course behave as they do. On synthetic data where the truth is known, you derive and then measure the bias-variance decomposition for linear models, neural networks and ensembles of trees, compare classifiers with the Bayes error, verify the variance formula that explains bagging and random forests, and study the cross-validation, bootstrap and out-of-bag estimates of the test error *as estimators*, with their own bias and variance.

Whichever path you choose, the report follows the same pattern as the two previous ones – abstract, introduction, theory and methods, implementation and tests, results and discussion, conclusions and perspectives, the LLM declaration, references – and the following holds for all three:

- You may reuse your own codes from projects 1 and 2, or use the functionality of **Scikit-Learn**, **JAX**, **TensorFlow/Keras**, **PyTorch** and similar libraries. Say clearly which is which.
- The quality measures you used and tested in projects 1 and 2 should be used again where they apply: the MSE and the R^2 score, accuracy, the confusion matrix, precision, recall and the F_1 score, ROC curves and the area under them, cumulative gains curves, and cross-validation or an equivalent resampling method for model selection (Chapters 2, 3 and 5 of the lecture notes).
- Hyperparameters are chosen on training/validation data; the test data are used once, at the end. Seeds and the chosen hyperparameters are documented so that every number in the report can be reproduced.
- Where it is possible, link your problem and your results with the existing literature. Scientific articles which have used machine learning algorithms on the same problem are highly welcome; perhaps you can improve on a previous analysis.
- A critical assessment of the methods, with perspectives and recommendations, is part of every path.

If none of the three paths fits your interests, you can propose your own problem or data set – for example related to your own research – as long as it follows the same pattern (at least two of the methods of the course, the quality measures above, a comparison with the literature, a critical assessment). **Please write to us before you start** if you go that way.

What you should take away from this project

Beyond the figures and tables in your report, these are the insights we want you to be able to state, explain and defend after working through the project, whichever path you choose. Use them as a checklist when you write your conclusions.

1. **The method must fit the problem, not the other way round.** A network that learns the solution of a differential equation, a convolutional network that learns features from pixels, a random forest on tabular data and a polynomial fit of a smooth function are the right tools for different problems. The report should say *why* the methods you chose are reasonable for your problem, and what you lose by not choosing the others.
2. **A number without a baseline and an error bar is not a result.** Every accuracy, MSE or test error you quote should be compared with something – a trivial predictor, a linear model, a published benchmark, the analytical solution or the Bayes error – and should come with an estimate of its uncertainty (the standard error of a test accuracy, the spread over cross-validation folds or over seeds).
3. **Model selection and model assessment are different things.** The data used to choose hyperparameters cannot be used to report the final error. Cross-validation, a validation split and the out-of-bag estimate serve the first purpose; a test set that is used once serves the second.
4. **Flexibility is paid for in variance.** Deep trees, wide networks, high polynomial degrees and finely resolved grids all reduce the bias and raise the variance; regularization – a penalty, early stopping, pruning, averaging, or a coarser model – trades them. You should be able to point to this trade-off in your own figures.
5. **Averaging removes variance, not bias.** Bagging and random forests improve on a single tree because a tree is unstable; averaging a biased model leaves the bias where it was. Boosting, and more expressive models, are the routes that attack the bias.
6. **Verification comes before results.** A finite-difference solution against the analytical one, a network against Scikit-Learn or PyTorch, a measured variance against a formula, a gradient against automatic differentiation: the tests that make an implementation trustworthy belong in the report.
7. **Reproducibility is part of the science.** Seeds, the hyperparameters you settled on, a `README` file that lets us rerun your notebooks, and citations of the data sets, the software and the literature you used.

Path A: Solving differential equations with neural networks

This path is tailored to those of you who are interested in studying differential equations and may have followed courses on their numerical solution. It is a field with large scientific interest, spanning from studies of turbulence in fluid mechanics and meteorology to the solution of quantum mechanical systems.

We assume that you have some background in the solution of partial differential equations using finite-difference schemes. We will study the solution of the diffusion equation in one dimension using a standard explicit scheme and neural networks to solve the same equation. For the explicit scheme you can study for example chapter 10 of the lecture notes in Computational Physics, FYS3150/4150, at https://github.com/CompPhysics/ComputationalPhysics/blob/master/doc/Lectures/lectures2015.pdf, or alternative sources from courses like MAT-MEK4270. For the solution of ordinary and partial differential equations using neural networks, the lectures of weeks 42 and 43 of this course (see https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week42 and https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial/week43) and Chapter 9 of the lecture notes (https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/chapter9.html) are highly recommended; as further reading you can use the textbook by Yadav, Yadav and Kumar, *An Introduction to Neural Network Methods for Differential Equations*, Springer 2015 (see https://www.springer.com/gp/book/9789401798150), the article by Lagaris, Likas and Fotiadis, IEEE Transactions on Neural Networks **9**, 987 (1998), which introduced the trial-function approach

with hard-coded conditions, and the article by Raissi, Perdikaris and Karniadakis, Journal of Computational Physics **378**, 686 (2019), which introduced physics-informed neural networks; see also https://maziarraissi.github.io/PINNs/.

For the machine learning part you can use your own code from project 2 or the functionality of for example **TensorFlow/Keras**, **PyTorch** or **JAX**. Whatever you use, the derivatives of the network output with respect to its inputs x and t, which enter the differential equation, should be computed by automatic differentiation (Section 4.14 of the lecture notes), not by finite differences.

Alternative differential equations

Note that you can replace the one-dimensional diffusion equation discussed below with other sets of either ordinary differential equations or partial differential equations – the wave equation, the Poisson equation of week 43, a system of coupled ordinary differential equations from mechanics or chemistry, or the eigenvalue problem for a symmetric matrix formulated as an ordinary differential equation (Yi, Fu and Zhou, Computers and Mathematics with Applications **47**, 1155 (2004)). Please discuss such a change with us at the lab before you start; the structure of parts a)–e) below should be kept.

Path A, part a): Setting up the problem

The physical problem can be that of the temperature in a rod of length $L = 1$, held at zero temperature at both ends. We are looking at the one-dimensional problem

$$\frac{\partial^2 u(x,t)}{\partial x^2} = \frac{\partial u(x,t)}{\partial t}, \qquad t > 0, \quad x \in [0, L],$$

or

$$u_{xx} = u_t,$$

with the initial condition, i.e. the condition at $t = 0$,

$$u(x, 0) = \sin(\pi x), \qquad 0 < x < L,$$

with $L = 1$ the length of the x-region of interest, and the boundary conditions

$$u(0, t) = 0, \qquad u(L, t) = 0, \qquad t \geq 0.$$

The function $u(x, t)$ can be the temperature of the rod. As time increases the initial profile decays towards the stationary state $u = 0$.

We will limit ourselves to the so-called explicit forward Euler algorithm, with a forward formula for the time derivative and a centered difference in space,

$$u_t \approx \frac{u(x, t + \Delta t) - u(x, t)}{\Delta t} = \frac{u(x_i, t_j + \Delta t) - u(x_i, t_j)}{\Delta t},$$

and

$$u_{xx} \approx \frac{u(x + \Delta x, t) - 2u(x, t) + u(x - \Delta x, t)}{\Delta x^2} = \frac{u(x_i + \Delta x, t_j) - 2u(x_i, t_j) + u(x_i - \Delta x, t_j)}{\Delta x^2}.$$

Write down the algorithm and the equations you need to implement. Find also the analytical solution to the problem (separation of variables), and discuss the stability condition of the explicit scheme.

Path A, part b): The explicit scheme

Implement the explicit scheme and perform tests of the solution for $\Delta x = 1/10$ and $\Delta x = 1/100$, using a Δt as dictated by the stability limit of the explicit scheme. The stability criterion requires that $\Delta t/\Delta x^2 \leq 1/2$.

Study the solutions at two time points t_1 and t_2, where $u(x, t_1)$ is smooth but still significantly curved and $u(x, t_2)$ is close to the stationary state, and compare with the analytical solution: plot the absolute error as a function of x at both times, and the maximal error as a function of Δx.

Path A, part c): Neural networks

Study now the lecture notes on solving ODEs and PDEs with neural networks (Chapter 9 and weeks 42 and 43) and use either your own code from project 2 or the functionality of TensorFlow/Keras, PyTorch or JAX to solve the same equation as in part b). Include both

- **hard-coded** boundary and initial conditions, where a trial function of the form

$$u_{\text{trial}}(x, t) = (1 - t)\,\sin(\pi x) + x(1 - x)\,t\,N(x, t; \boldsymbol{\theta})$$

  (or a similar construction) satisfies the conditions identically and the network N is trained to make the residual $u_{xx} - u_t$ small at a set of collocation points; and
- **soft** conditions, the physics-informed neural network (PINN), where the network itself is the solution and the cost function is the sum of the squared residual of the equation at interior points and the squared violations of the initial and boundary conditions at points on the boundary of the domain, with weights for the different terms.

Discuss your results and compare them with the explicit scheme and with the analytical solution, at the same two time points as in part b) and as a function of x and t over the whole domain. Discuss the pros and cons of the two ways of imposing the conditions, and the differences between the finite-difference and the neural-network approaches: what is the unknown in each case, where does the error come from, what is the computational cost, and how does each of them behave when you evaluate the solution at points that were not used in the computation?

Path A, part d): Neural network complexity

Here we study the stability of the results as functions of the number of hidden nodes, the number of layers and the activation functions of the hidden layers. Increase the number of hidden nodes and layers in order to see if this improves your results. Try also different activation functions for the hidden layers, such as tanh, the sigmoid, ReLU and others, and explain which of them can be used at all for a second-order equation. Study also the number and the placement of the collocation points, the weighting of the terms of the PINN cost function, and the role of the optimizer and the learning rate. Discuss your results, including the computational cost of each configuration.

Path A, part e): Critical assessment

Finally, present a critical assessment of the methods you have studied and discuss the potential for solving differential equations with machine learning methods: when does a neural network solver make sense compared with a classical scheme, what does it cost, what guarantees does it give, and where is the field going (inverse problems, high dimensions, data-driven terms in the equations)?

Summary of requirements for path A

Required implementation: the explicit scheme for the diffusion equation with the analytical solution as a test; a neural-network solver with hard-coded conditions; a PINN with soft conditions; derivatives of the network with respect to x and t by automatic differentiation; a systematic scan over the network architecture.

Required analysis: the error of each method against the analytical solution at two times and over the domain; the dependence on Δx for the scheme and on the architecture, the activation function, the collocation points and the training for the networks; a comparison of computational cost; the critical assessment of part e).

Path B: A classification study with several methods

This path follows the structure of a typical applied machine-learning study: one classification problem, several methods, one honest comparison. The methods must include **at least three** of those we have discussed during the semester – logistic regression, feed-forward neural networks, convolutional neural networks, recurrent neural networks, decision trees, bagging, random forests, boosting and gradient boosting – and among the three there should be **at least one neural network** (feed-forward or convolutional) and **at least one tree-based ensemble** (random forest, gradient boosting or XGBoost). A natural triple for an image data set is gradient boosting, a feed-forward neural network and a convolutional network; for a tabular data set it could be logistic regression, a random forest and a gradient-boosting method. Logistic regression is cheap and we recommend that you always include it as the baseline, as in project 2. Support vector machines (Chapter 6 of the lecture notes) are welcome as an additional method, although they have not been covered in the lectures.

The default data set: Fashion-MNIST

Our default proposal is **Fashion-MNIST** (Xiao, Rasul and Vollgraf, arXiv:1708.07747 (2017)), see https://github.com/zalandoresearch/fashion-mnist. It consists of 70 000 grey-scale images of 28×28 pixels of clothing items from the Zalando catalogue, in ten classes (T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot), with the canonical split into 60 000 training and 10 000 test images. It has exactly the format of MNIST, so your code from project 2 runs on it unchanged, but it is a harder problem: on the benchmark page of the repository, logistic regression on the raw pixels reaches an accuracy of roughly 0.84, random forests and gradient boosting roughly 0.87–0.89, a feed-forward network roughly 0.88–0.90, and convolutional networks between 0.91 and 0.94 depending on their size. The classes shirt, T-shirt/top, pullover and coat are confused with each other by every method, which makes the confusion matrix genuinely informative.

The data can be read with the loaders of the libraries, for example

```
# TensorFlow/Keras
from tensorflow.keras.datasets import fashion_mnist
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
print(x_train.shape, y_train.shape, x_test.shape, y_test.shape)
# (60000, 28, 28) (60000,) (10000, 28, 28) (10000,)

# PyTorch
from torchvision.datasets import FashionMNIST
train = FashionMNIST(root="data", train=True, download=True)
test = FashionMNIST(root="data", train=False, download=True)
```

or downloaded as the four compressed files described on the GitHub page. As in project 2, flatten the images to vectors of 784 features and scale the pixel values to $[0, 1]$ for the feed-forward network, the logistic regression and the tree-based methods (trees do not need the scaling, but it does no harm), and keep the 28×28 shape for the convolutional network. Use the canonical split as intended: all training, model selection and hyperparameter tuning on the training data (a validation split or cross-validation), the test set once at the end. If you are limited by computing power, you may tune on a subset of the training data, but the numbers you report should be for the full test set. Cite the data set.

Other data sets

You can replace Fashion-MNIST with a data set of your own choice, preferably one that has been studied in the literature so that you have results to compare with. Possibilities are CIFAR-10 (`https://www.cs.toronto.edu/~kriz/cifar.html`, 60 000 colour images in ten classes, harder than Fashion-MNIST and more demanding for the convolutional network), tabular classification sets from the UCI repository (`https://archive.ics.uci.edu/ml/index.php`; for example the covertype, adult, wine-quality or spambase data, where the tree-based methods are usually the strongest and the convolutional network is replaced by a feed-forward one), sequence data where a recurrent network is natural (Chapter 11), or data from your own field of study. Kaggle (`https://www.kaggle.com/datasets`) hosts many documented data sets with published analyses. If you use your own data, write to us before you start. Binary classification problems are allowed if the data set is rich enough; the quality measures of Chapter 5 then apply directly.

Path B, part a): The data

Describe how the data were produced and place them in a proper context: what is a sample, what are the features, what are the classes and how balanced are they? Show examples. Describe the preprocessing (scaling, flattening or reshaping, one-hot encoding of the labels) and the split into training, validation and test data, and explain why you split as you do. Cite the source of the data and the articles that have analyzed them before.

Path B, part b): The methods

Explain the basics of the methods you have chosen to work with. This is the theory part of your report: the model, its cost function, how it is trained, which hyperparameters it has and what they do. For a tree-based ensemble this means the splitting criterion (Gini index or entropy), bagging and the random feature subset of a forest, or the additive model and the loss of a boosting method (Chapter 7); for the networks the architecture, the activation functions, the softmax output and the cross entropy (Chapters 8 and 10). Explain also the quality measures you will use (Section 5.12 of the lecture notes).

Path B, part c): Implementation and tests

Describe your algorithms and their implementation, and the tests you have performed. Say which parts are your own code (from project 2 or new) and which parts use Scikit-Learn, TensorFlow/Keras, PyTorch, XGBoost or other libraries. For each method, describe how you searched for the hyperparameters – the learning rate and the regularization for the networks, the depth, number of trees, number of features per split and learning rate for the ensembles – using cross-validation or a validation set (Chapter 3 and week 44), and present the search as heat maps or curves rather than as a list of runs. Report the training times and the number of parameters of each model. Use seeds.

Path B, part d): Results

Present your results and findings for all methods on the *same* test data: the accuracy with its standard error, the confusion matrix, the per-class precision, recall and F_1 score, and one-versus-rest ROC curves with their areas (and cumulative gains curves if you find them useful). Compare the methods with each other, with the logistic-regression baseline, and with the published benchmarks or articles for your data set. Show which classes are confused and look at the misclassified samples; for the trees show the feature importances, for the convolutional network the learned filters or a few feature maps. Discuss whether the differences you see are larger than the uncertainties.

Path B, part e): Critical assessment

Finally, present a critical assessment of the methods you have studied and link your results with the existing literature. Which method is best for your data, at what cost in computing time, data preparation and interpretability? Which would you recommend to somebody facing a similar problem, and what would you try next?

Summary of requirements for path B

Required implementation: at least three methods of the course, including a neural network and a tree-based ensemble, applied to one classification data set that is not MNIST; a systematic hyperparameter search on training or validation data for each method; the quality measures of Chapter 5 (accuracy, confusion matrix, precision, recall, F_1, ROC and AUC).

Required analysis: the data in context; the theory of each method; a comparison of all methods on the same test set with uncertainties; a comparison with the published literature or benchmarks; the misclassified classes and samples; the critical assessment of part e).

Path C: The statistics and mathematics behind the methods

This path is for those of you who want to understand *why* the methods of the course behave as they do, rather than to apply them to a new data set. You will work mostly with synthetic data, where the true function, the noise and the Bayes error are known exactly, so that every quantity we have talked about during the semester – bias, variance, the expected test error, the correlation between the members of an ensemble, the bias of an error estimator – can be *measured* and compared with what the mathematics predicts. Derivations and controlled numerical experiments carry equal weight here; the report should read as a small study in statistical learning, with the theory in the methods section and the experiments confirming (or refining) it in the results.

The data we propose are

- for regression, a known function with Gaussian noise of known variance σ^2, for example the one-dimensional Runge function $f(x) = 1/(1 + 25x^2)$ of projects 1 and 2, or the two-dimensional Runge function or Franke's function; you draw as many independent training sets of size n as you need, and a large independent test set;
- for classification, a mixture of two (or more) Gaussian distributions in p dimensions with known means and covariances. For two classes with a common covariance matrix $\boldsymbol{\Sigma}$ and means $\boldsymbol{\mu}_0$, $\boldsymbol{\mu}_1$, the Bayes classifier is linear and the Bayes error is $\Phi(-\Delta/2)$, with $\Delta^2 = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$ the Mahalanobis distance and Φ the standard normal cumulative distribution; with different covariances or with several components per class the Bayes boundary is curved and the Bayes error can be computed numerically from the known densities.

You may add one real data set at the end to see how much of what you found carries over, but the heart of this path is the controlled experiments.

Path C, part a): Theory

This part is the mathematical backbone of the report. Derive, with the lecture notes as your guide:

1. the decomposition of the expected test error into bias, variance and irreducible noise (Section 2.9 of the lecture notes), and how each term is estimated from many training sets or from the bootstrap;
2. the expectation and the covariance of the OLS and Ridge estimators (Sections 3.7 and 3.8), the bias of Ridge as a function of λ in terms of the singular values of the design matrix, and the resulting bias and variance of the Ridge *prediction*;
3. the Bayes classifier and the Bayes error for the Gaussian mixture above; logistic regression as maximum likelihood (Chapter 5), the Fisher information and the asymptotic covariance of its coefficients;
4. the variance of an average of B predictors with pairwise correlation ρ, Eq. (7.16) of the lecture notes, $\mathrm{var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$, and why bagging and random forests act on different terms of it;
5. what the k-fold cross-validation, leave-one-out, bootstrap, .632 and out-of-bag estimates of the test error are, which quantity each of them estimates, and why one expects leave-one-out to have small bias and large variance, the plain bootstrap estimate to be optimistic, and the out-of-bag estimate to be nearly unbiased (Sections 2.10–2.11, 3.14 and 7.9).

Path C, part b): Linear models and neural networks on the regression data

On the regression data, for polynomial OLS and Ridge of increasing degree and for a feed-forward neural network of increasing width (your code from project 2 or a library):

- measure bias^2, variance and total test error at a grid of test points by fitting the model to many independent training sets, and compare with the bootstrap estimates computed from a *single* training set, as in project 1;
- for OLS and Ridge compare the measured bias and variance with the closed-form expressions of part a) at fixed design, and show how the Ridge path trades the two as λ varies;
- for the network show the same trade-off as a function of the width, of the number of epochs (early stopping) and of the L_2 penalty, and compare with the polynomial curve;
- (optional) study OLS when the number of parameters p crosses the number of training points n: use the minimum-norm solution (the pseudoinverse) for $p > n$ and show the peak of the test error at $p \approx n$ – the so-called double descent – and how Ridge removes it.

Path C, part c): Classifiers against the Bayes error

On the Gaussian-mixture data:

- fit logistic regression for increasing training sizes n and compare its test error with the Bayes error; compare the empirical covariance of the fitted coefficients over many training sets with the inverse Fisher information and with the bootstrap;
- repeat with a feed-forward neural network (your project-2 code) and with a random forest: how close to the Bayes rate do they get, how fast with n, and what do their decision boundaries look like compared with the Bayes boundary? Make the Bayes boundary curved (different covariances per class, or several components) so that the linear model has a bias;

- compare the predicted probabilities of the three models with the true posterior probabilities (reliability or calibration curves) – a model can have a good accuracy and badly calibrated probabilities;
- study what class imbalance does to the accuracy, the balanced accuracy, precision, recall, the F_1 score and the Matthews correlation coefficient (Section 5.12), by changing the class priors of the mixture.

Path C, part d): Ensembles of trees and Eq. (7.16)

On the regression data (and, if you wish, on the classification data):

- fit many deep regression trees to bootstrap samples of many independent training sets, and measure σ^2 (the variance of a single tree), ρ (the mean correlation between two trees of the same ensemble) and the variance of the average of B trees; compare with Eq. (7.16) with the measured σ^2 and ρ – no fitting;
- on data with several features, show how the random feature subset of a random forest lowers ρ (and raises σ^2) as the number of features per split m decreases, and what this does to the test error;
- decompose the test error of a single deep tree, of bagging and of a random forest into bias and variance, and show that the ensembles change the variance but not the bias; (optional, after week 47) add gradient boosting and show that it changes the bias;
- compare the out-of-bag estimate of the test error with cross-validation on the same ensemble.

Path C, part e): The error estimators themselves

Here you turn the tools of model assessment on themselves. With the true test error available from a large independent test set, treat k-fold cross-validation (for example $k = 5$, 10 and n), the bootstrap estimate, the .632 estimate and the out-of-bag estimate as random variables, by repeating the whole procedure on many independent training sets, and measure the bias and the variance of each estimator as functions of the training size n and of the model flexibility, for at least one regression and one classification model. Study also:

- the one-standard-error rule for choosing a hyperparameter from a cross-validation curve (Section 3.14), and how often it picks the model with the smallest true test error;
- what happens when the same cross-validation is used both to choose the hyperparameter and to report the error – the optimism of the selected minimum – and how nested cross-validation removes it.

Path C, part f): Critical assessment

Finally, present a critical assessment: which of the predictions of the theory held up in your experiments, where did they break down and why (finite n, non-linear models, dependence between folds, ...), and which of your findings carry over to real data, where none of the true quantities is known. What would you now do differently when assessing a model on a real data set?

Summary of requirements for path C

Required implementation: generators for the regression and the Gaussian-mixture data with the true function, the noise and the Bayes error available; the bias-variance measurement from many training sets and from the bootstrap; OLS/Ridge, logistic regression, a feed-forward network and bagged trees/random forests (your own codes or libraries); the resampling estimators of part e).

Required analysis: the derivations of part a) in the theory section; the comparison of every measured quantity with its predicted value (closed-form bias and variance, Bayes error, Fisher information, Eq. (7.16), the expected bias of the estimators), with uncertainties; the critical assessment of part f). The items marked optional are not required.

Background literature

- The lecture notes of the course at https://educationalmaterialuio.github.io/MachineLearningUiO/doc/LectureNotes/_build/html/: Chapters 2 and 3 (statistics, resampling, linear regression), Chapter 5 (logistic regression and the quality measures of a classifier), Chapter 7 (decision trees, bagging, random forests and boosting), Chapter 8 (neural networks), Chapter 9 (differential equations with neural networks), Chapter 10 (convolutional networks) and Chapter 11 (recurrent networks).
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, Springer, at https://hastie.su.domains/ElemStatLearn/; chapters 7 (model assessment and selection), 9 (trees), 10 (boosting) and 15 (random forests) are the ones for paths B and C.
- Goodfellow, Bengio and Courville, *Deep Learning*, at https://www.deeplearningbook.org/, chapters 6–9.
- Raschka et al. at https://sebastianraschka.com/blog/2022/ml-pytorch-book.html, chapters 3, 6, 7, 11–14.
- For path A: Lagaris, Likas and Fotiadis, IEEE Trans. Neural Networks **9**, 987 (1998); Raissi, Perdikaris and Karniadakis, J. Comput. Phys. **378**, 686 (2019); Yadav, Yadav and Kumar, *An Introduction to Neural Network Methods for Differential Equations*, Springer (2015).
- For path B: Xiao, Rasul and Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747 (2017), and the benchmark table at https://github.com/zalandoresearch/fashion-mnist.
- For path C: Breiman, *Bagging predictors*, Machine Learning **24**, 123 (1996) and *Random forests*, Machine Learning **45**, 5 (2001); Efron and Tibshirani, *An Introduction to the Bootstrap*, Chapman and Hall (1993); Belkin, Hsu, Ma and Mandal, *Reconciling modern machine-learning practice and the classical bias-variance trade-off*, PNAS **116**, 15849 (2019), for the optional double-descent item.

Introduction to numerical projects

Here follows a brief recipe and recommendation on how to answer the various questions when preparing your answers.

- Give a short description of the nature of the problem and the eventual numerical methods you have used.
- Describe the algorithm you have used and/or developed. Here you may find it convenient to use pseudocoding. In many cases you can describe the algorithm in the program itself.
- Include the source code of your program. Comment your program properly. You should have the code at your GitHub/GitLab link. You can also place the code in an appendix of your report.
- If possible, try to find analytic solutions, or known limits in order to test your program when developing the code. In this project the analytical solution of the diffusion equation (path A), the published benchmarks (path B) and the closed-form bias, variance and Bayes error (path C) are exactly such tests.
- Include your results either in figure form or in a table. Remember to label your results. All tables and figures should have relevant captions and labels on the axes.

- Try to evaluate the reliability and numerical stability/precision of your results. If possible, include a qualitative and/or quantitative discussion of the numerical stability, eventual loss of precision etc.
- Try to give an interpretation of your results in your answers to the problems.
- Critique: if possible include your comments and reflections about the exercise, whether you felt you learnt something, ideas for improvements and other thoughts you've made when solving the exercise. We wish to keep this course at the interactive level and your comments can help us improve it.
- Try to establish a practice where you log your work at the computer lab. You may find such a logbook very handy at later stages in your work, especially when you don't properly remember what a previous test version of your program did. Here you could also record the time spent on solving the exercise, various algorithms you may have tested or other topics which you feel worthy of mentioning.

Format for electronic delivery of report and programs

The preferred format for the report is a PDF file. You can also use DOC or postscript formats or an IPython notebook file. As programming language we prefer that you choose between C/C++, Fortran2008, Julia or Python. The following prescription should be followed when preparing the report:

- Use Canvas to hand in your projects, log in at https://www.uio.no/english/services/it/education/canvas/ with your normal UiO username and password.
- Upload **only** the report file or the link to your GitHub/GitLab or similar type of repository! For the source code file(s) you have developed please provide us with your link to your GitHub/GitLab or similar domain. The report file should include all of your discussions and a list of the codes you have developed. Do not include library files which are available at the course homepage, unless you have made specific changes to them.
- In your GitHub/GitLab or similar repository, please include a folder which contains selected results. These can be in the form of output from your code for a selected set of runs and input parameters.

Finally, we encourage you to collaborate. Optimal working groups consist of 2–3 students. You can then hand in a common report.

Software and needed installations

If you have Python installed (we recommend Python 3) and you feel pretty familiar with installing different packages, we recommend that you install the following Python packages via `pip` as

```
pip install numpy scipy matplotlib ipython jupyter scikit-learn
pip install pandas seaborn
pip install jax jaxlib
```

and, depending on your path, `tensorflow` or `torch` (and `torchvision` for the data loaders) for the neural networks of paths A and B, and `xgboost` if you choose it for the gradient boosting of path B. For Python 3, replace `pip` with `pip3` where needed.

For OSX users we recommend also, after having installed Xcode, to install `brew`. Brew allows for a seamless installation of additional software via for example `brew install python3`. For Linux users, with its variety of distributions like for example the widely popular Ubuntu distribution, you can use `pip` as well and simply install Python as `sudo apt-get install python3` etc.

If you don't want to install various Python packages with their dependencies separately, we recommend Anaconda, an open source distribution of the Python and R programming languages for large-scale data processing, predictive analytics, and scientific computing, that aims to simplify package management and deployment. Package versions are managed by the package management system `conda`.

Popular software packages written in Python for ML are Scikit-learn, JAX, TensorFlow, PyTorch and Keras. These are all freely available at their respective GitHub sites. They encompass communities of developers in the thousands or more, and the number of code developers and contributors keeps increasing.